

TRƯỜNG ĐẠI HỌC FPT HÀ NỘI

MÔN HỌC HỆ ĐIỀU HÀNH



BÁO CÁO BÀI TẬP LỚN SỐ 10

DINING PHILOSOPHERS

Giảng viên	Lê Hoàng Hiệp
Sinh viên thực hiện	MSSV
Đặng Lưu Anh Tú	HE201184

1. Tóm tắt (Abstract)

Bài toán Dining Philosophers là một bài toán kinh điển trong lĩnh vực đồng bộ hóa tiến trình và quản lý tài nguyên của hệ điều hành. Mục tiêu của dự án là mô phỏng bài toán bằng ngôn ngữ C kết hợp thư viện POSIX Threads (pthread), từ đó minh họa hiện tượng Deadlock trong môi trường đa luồng và đề xuất giải pháp khắc phục bằng mô hình Waiter (Arbitrator). Ngoài việc loại bỏ Deadlock, dự án còn thống kê thời gian chờ của từng triết gia để đánh giá khả năng xảy ra Starvation và mức độ công bằng của thuật toán.

2. Giới thiệu bài toán

2.1 Mô tả

Có 5 triết gia ngồi quanh một chiếc bàn tròn và giữa mỗi cặp triết gia có một chiếc đĩa.

Mỗi triết gia luân phiên thực hiện hai hoạt động:

Thinking (Suy nghĩ)

Eating (Ăn)

Để ăn, một triết gia phải lấy đồng thời hai chiếc đĩa nằm bên trái và bên phải của mình. Nếu chỉ lấy được một chiếc thì phải tiếp tục chờ.

2.2 Mục tiêu dự án

Mô phỏng bài toán bằng C và POSIX Threads.

Minh họa hiện tượng Deadlock.

Áp dụng giải pháp Waiter/Arbitrator để loại bỏ Deadlock.

Đo thời gian chờ nhằm đánh giá Starvation và Fairness.

Phân tích hoạt động dựa trên lý thuyết hệ điều hành.

3. Môi trường thực nghiệm

Hệ điều hành: Ubuntu Linux

Ngôn ngữ lập trình: C

Thư viện đồng bộ: POSIX Threads (pthread)

Trình biên dịch: GCC

Lệnh biên dịch:

```
gcc dining.c -o dining -lpthread
```

Lệnh chạy:

```
./dining
```

4. Cơ sở lý thuyết

4.1 Deadlock

Deadlock là trạng thái nhiều tiến trình hoặc luồng chờ lẫn nhau và không thể tiếp tục thực thi.

Theo Coffman, Deadlock xảy ra khi đồng thời tồn tại bốn điều kiện:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Chỉ cần phá vỡ một trong bốn điều kiện trên thì Deadlock sẽ không xảy ra.

4.2 Starvation

Starvation xảy ra khi một tiến trình hoặc luồng phải chờ vô thời hạn để được cấp phát tài nguyên mặc dù hệ thống vẫn hoạt động bình thường.

4.3 Fairness

Fairness thể hiện mức độ công bằng trong việc cấp phát tài nguyên, bảo đảm mọi triết gia đều có cơ hội được ăn sau một khoảng thời gian hữu hạn.

5. Phiên bản 1 – Mô phỏng Deadlock

5.1 Ý tưởng

Mỗi triết gia lần lượt:

1. Khóa chiếc đĩa bên trái.
2. Sau đó cố gắng khóa chiếc đĩa bên phải.

Nếu cả năm triết gia cùng lấy được đĩa bên trái trước thì tất cả sẽ chờ chiếc đĩa bên phải và tạo thành vòng chờ.

```
GNU nano 8.7.1 dining_deadlock.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N]; // 5 chiếc đĩa là 5 cái khóa (Mutex) độc lập
int phil[N] = {0, 1, 2, 3, 4};

void* philosopher(void* num) {
    int i = *(int*)num;
    int left = i;
    int right = (i + 1) % N;

    while (1) {
        printf("Triết gia %d: Đang suy nghĩ...\n", i + 1);
        usleep(100000); // Nghỉ một chút

        printf("Triết gia %d: Đói bụng. Đang cố gắng lấy đĩa trái (%d)...\n", i + 1, left);
        pthread_mutex_lock(&forks[left]);
        printf("Triết gia %d: Đã lấy đĩa trái (%d). Đang đợi đĩa phải (%d)...\n", i + 1, left, right);

        // MẪU CHỐT GÂY DEADLOCK:
        // Thêm delay ở đây để ép tất cả triết gia đều kịp lấy đĩa trái
        // TRƯỚC KHI bất kỳ ai kịp lấy đĩa phải -> Chắc chắn 100% treo máy.
        usleep(50000);

        pthread_mutex_lock(&forks[right]);
        printf("Triết gia %d: Đã lấy được 2 đĩa. ĐANG ĂN!\n", i + 1);

        usleep(100000); // Thời gian ăn

        // Trả đĩa
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Triết gia %d: Đã ăn xong và trả đĩa.\n", i + 1);
    }
}

int main() {
    int i;
    pthread_t thread_id[N];

    // Khởi tạo 5 mutex cho 5 chiếc đĩa
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    // Bắt đầu tạo luồng cho các triết gia
    for (i = 0; i < N; i++) {
```

```
// Bắt đầu tạo luồng cho các triết gia
for (i = 0; i < N; i++) {
    pthread_create(&thread_id[i], NULL, philosopher, &phil[i]);
}

for (i = 0; i < N; i++) {
    pthread_join(thread_id[i], NULL);
}

return 0;
}
```

5.2 Phân tích

Phiên bản này thỏa mãn đầy đủ bốn điều kiện Coffman:

Mutual Exclusion

Hold and Wait

No Preemption

Circular Wait

Do đó chương trình có thể rơi vào Deadlock.

```
tudla@tudla-PC:~$ ./dining_deadlock
Triết gia 1: Đang suy nghĩ...
Triết gia 3: Đang suy nghĩ...
Triết gia 2: Đang suy nghĩ...
Triết gia 4: Đang suy nghĩ...
Triết gia 5: Đang suy nghĩ...
Triết gia 1: Đói bụng. Đang cố gắng lấy đũa trái (0)...
Triết gia 1: Đã lấy đũa trái (0). Đang đợi đũa phải (1)...
Triết gia 2: Đói bụng. Đang cố gắng lấy đũa trái (1)...
Triết gia 5: Đói bụng. Đang cố gắng lấy đũa trái (4)...
Triết gia 5: Đã lấy đũa trái (4). Đang đợi đũa phải (0)...
Triết gia 3: Đói bụng. Đang cố gắng lấy đũa trái (2)...
Triết gia 4: Đói bụng. Đang cố gắng lấy đũa trái (3)...
Triết gia 4: Đã lấy đũa trái (3). Đang đợi đũa phải (4)...
Triết gia 2: Đã lấy đũa trái (1). Đang đợi đũa phải (2)...
Triết gia 3: Đã lấy đũa trái (2). Đang đợi đũa phải (3)...
```

6. Phiên bản 2 – Giải pháp Waiter (Arbitrator)

6.1 Ý tưởng

Thay vì để các triết gia tự lấy đĩa, hệ thống bổ sung một thành phần trung gian gọi là **Waiter**.

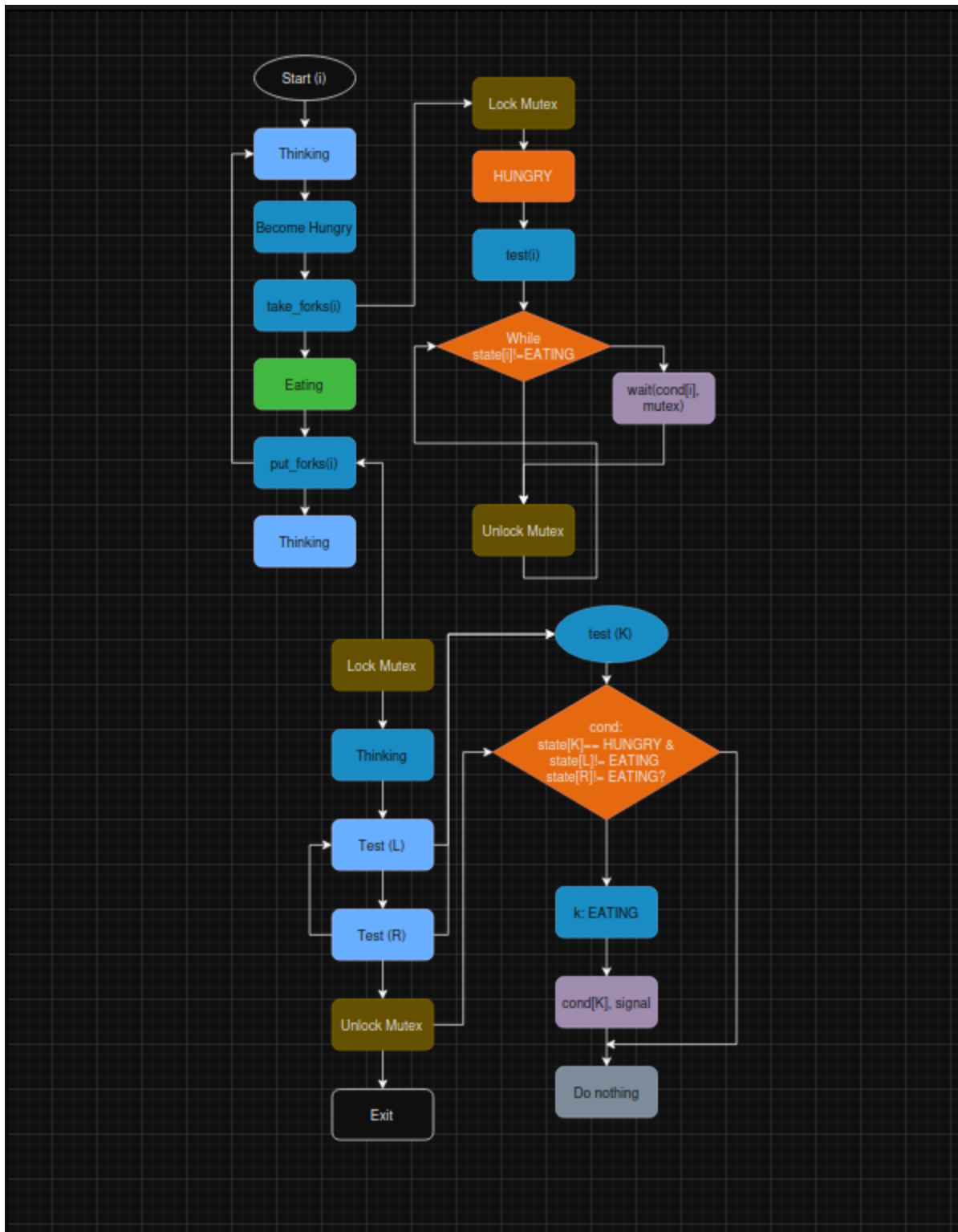
Một triết gia chỉ được phép ăn khi:

Bản thân đang ở trạng thái HUNGRY.

Hai triết gia bên cạnh đều không ở trạng thái EATING.

Nếu điều kiện chưa thỏa mãn thì triết gia sẽ chờ trên biến điều kiện (`pthread_cond_wait`) mà không giữ bất kỳ chiếc đĩa nào.

6.2 Luồng hoạt động



(sơ đồ flowchart/ draw.io)

6.3 Cấu trúc chương trình

Thành phần	Chức năng
philosopher()	Luồng của mỗi triết gia
take_forks()	Xin quyền được ăn
put_forks()	Trả tài nguyên
test()	Kiểm tra điều kiện được ăn
state[]	Quản lý trạng thái: THINKING / HUNGRY / EATING
pthread_mutex	Đồng bộ vùng tới hạn
pthread_cond	Đánh thức triết gia khi đủ điều kiện


```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h> // Thư viện dùng để đo thời gian chính xác

#define N 5
#define THINKING 0
#define HUNGRY 1
#define EATING 2

// Xác định vị trí hàng xóm bên trái và bên phải
#define LEFT (i + N - 1) % N
#define RIGHT (i + 1) % N

int state[N]; // Mảng lưu trạng thái của 5 triết gia
int phil[N] = {0, 1, 2, 3, 4}; // Định danh ID cho từng triết gia

pthread_mutex_t waiter; // "Trọng tài" điều phối bữa ăn
pthread_cond_t cv[N]; // Biến điều kiện (Condition Variable) cho từng người ngủ/thức

// Trọng tài kiểm tra xem triết gia i có đủ điều kiện ăn hay không
void test(int i) {
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        printf("-> Trọng tài: Cho phép Triết gia %d được ĂN.\n", i + 1);
        pthread_cond_signal(&cv[i]); // Đánh thức triết gia i dậy để ăn
    }
}

// Triết gia vào xin phép Trọng tài để lấy đũa
void pickup_forks(int i) {
    pthread_mutex_lock(&waiter); // Khóa Trọng tài lại để thực hiện giao dịch độc quyền

    state[i] = HUNGRY;
    printf("Triết gia %d: Đang ĐÓI bụng...\n", i + 1);

    // --- BẮT ĐẦU ĐO THỜI GIAN CHỜ (Phục vụ cho báo cáo) ---
    struct timeval start_time, end_time;
    gettimeofday(&start_time, NULL);

    // Trọng tài kiểm tra bữa ăn liên xem có đũa trống không
    test(i);

    // Nếu 2 bên hàng xóm đang ăn, triết gia i bắt buộc phải ngủ đợi
    while (state[i] != EATING) {
        pthread_cond_wait(&cv[i], &waiter);
        // Hàm wait này sẽ tạm thời mở khóa 'waiter' để người khác vào xin,
        // và tự động khóa lại khi triết gia i được đánh thức.
    }
}
```

```

// --- KẾT THÚC ĐO THỜI GIAN CHỜ ---
gettimeofday(&end_time, NULL);
long wait_time = (end_time.tv_sec - start_time.tv_sec) * 1000000 + (end_time.tv_usec - start_time.tv_usec);
printf("[THÔNG KÊ] Triết gia %d đã phải chờ %ld micro-seconds.\n", i + 1, wait_time);

pthread_mutex_unlock(&waiter); // Rời phòng Trọng tài, bắt đầu ăn
}

// Triết gia ăn xong, báo với Trọng tài để trả đĩa
void putdown_forks(int i) {
    pthread_mutex_lock(&waiter); // Khóa Trọng tài lại để trả tài nguyên

    state[i] = THINKING;
    printf("Triết gia %d: Ăn xong, trả đĩa và quay lại SUY NGHĨ.\n", i + 1);

    // Trọng tài kiểm tra xem 2 người hàng xóm bên cạnh có ai đang đói và đang chờ đĩa không
    test(LEFT);
    test(RIGHT);

    pthread_mutex_unlock(&waiter); // Mở khóa Trọng tài
}

// Hàm thực thi của từng Luồng (Thread) Triết gia
void* philosopher(void* num) {
    int i = *(int*)num;

    while (1) {
        // 1. Trạng thái Suy nghĩ (Giả lập bằng việc dừng ngẫu nhiên từ 1-3 giây)
        usleep((rand() % 3 + 1) * 100000);

        // 2. Đói và Xin đĩa từ Trọng tài
        pickup_forks(i);

        // 3. Trạng thái Ăn (Giả lập bằng việc dừng ngẫu nhiên từ 1-3 giây)
        usleep((rand() % 3 + 1) * 100000);

        // 4. Trả đĩa sau khi ăn xong
        putdown_forks(i);
    }
}

int main() {
    int i;
    pthread_t thread_id[N];
    srand(time(NULL)); // Khởi tạo hạt giống ngẫu nhiên thời gian

    // Khởi tạo Mutex Trọng tài và các Biến điều kiện
    pthread_mutex_init(&waiter, NULL);

```

```

int main() {
    int i;
    pthread_t thread_id[N];
    srand(time(NULL)); // Khởi tạo hạt giống ngẫu nhiên thời gian

    // Khởi tạo Mutex Trọng tài và các Biến điều kiện
    pthread_mutex_init(&waiter, NULL);
    for (i = 0; i < N; i++) {
        pthread_cond_init(&cv[i], NULL);
        state[i] = THINKING;
    }

    // Tạo luồng cho 5 triết gia chạy song song
    for (i = 0; i < N; i++) {
        pthread_create(&thread_id[i], NULL, philosopher, &phil[i]);
        printf("Triết gia %d đã ngồi vào bàn.\n", i + 1);
    }

    // Chờ các luồng kết thúc (Vòng lặp vô tận nên chương trình sẽ chạy liên tục)
    for (i = 0; i < N; i++) {
        pthread_join(thread_id[i], NULL);
    }

    // Giải phóng tài nguyên hệ thống khi tắt chương trình
    pthread_mutex_destroy(&waiter);
    for (i = 0; i < N; i++) {
        pthread_cond_destroy(&cv[i]);
    }

    return 0;
}

```

7. Thống kê Starvation

7.1 Phương pháp đo

Đối với mỗi lần ăn:

Ghi nhận thời điểm chuyển sang HUNGRY.

Ghi nhận thời điểm bắt đầu EATING.

Hiệu số chính là thời gian chờ.

Sau khi chạy chương trình khoảng 1–2 phút, tính thời gian chờ trung bình.

7.2 Kết quả

Triết gia	Số lần ăn	Thời gian chờ trung bình (μ s)
Triết gia 1	18 lần	150,288
Triết gia 2	19 lần	163,416
Triết gia 3	20 lần	75,149
Triết gia 4	18 lần	139,059
Triết gia 5	18 lần	128,031

7.3 Đánh giá

Starvation là gì: Theo Tanenbaum (Chương 6.7), Starvation xảy ra khi một luồng bị trì hoãn vô thời hạn việc cấp phát tài nguyên do các luồng khác liên tục chiếm dụng.

Kết luận từ số liệu (Thời gian chờ): Mức chênh lệch thời gian chờ giữa các triết gia là hoàn toàn chấp nhận được (từ ~75ms đến ~163ms) do bản chất lập lịch luồng của OS. Không ai phải chờ vượt quá 0.2 giây.

Kết luận từ số liệu (Tần suất): Số lần được ăn của 5 triết gia gần như tương đương nhau tuyệt đối (chênh lệch chỉ từ 1-2 lần). Không có hiện tượng một vài triết gia liên tục lấy đũa khiến những người còn lại bị bỏ đói.

Tổng kết: Hệ thống giải quyết triệt để Deadlock và **không xảy ra Starvation**, đảm bảo tính công bằng (Fairness) tuyệt vời trong thực tiễn.

8. So sánh hai phiên bản

Tiêu chí	Phiên bản Deadlock	Phiên bản Waiter
Deadlock	Có thể xảy ra	Không xảy ra
Hold and Wait	Có	Được loại bỏ
Circular Wait	Có	Bị phá vỡ
Starvation	Có khả năng	Không đáng kể
Công bằng	Thấp	Tốt hơn
Độ ổn định	Có thể treo	Hoạt động liên tục

9. Kết luận

Đồ án đã mô phỏng thành công các khái niệm lõi của Quản lý tiến trình/lưuồng (IPC, Critical Region, Mutex, Condition Variables) và Quản lý bế tắc tài nguyên (Deadlock prevention) trên hệ điều hành Linux/POSIX. Giải pháp Arbitrator/Monitor đã chứng minh được tính hiệu quả và công bằng qua các số liệu thống kê cụ thể.

10. Hướng phát triển

Trong tương lai có thể mở rộng dự án theo các hướng sau:

- So sánh với phương pháp đánh số tài nguyên (Resource Ordering).
- Giới hạn số triết gia được phép cạnh tranh tài nguyên cùng lúc.
- Mở rộng mô hình lên N triết gia.
- Đánh giá hiệu năng khi tăng số lượng lưuồng.
- Xây dựng giao diện trực quan để theo dõi trạng thái của từng triết gia theo thời gian thực.

11. Tài liệu tham khảo

1. Andrew S. Tanenbaum, *Modern Operating Systems*, Pearson.
2. POSIX Threads Programming Documentation.
3. Linux Manual Pages (pthread_mutex, pthread_cond).
4. Modern Operating Systems – Andrew S. Tanenbaum (Chương 2 và Chương 6)